

sudo 与最小特权授权

把“给 sudo 权限”改写成可审计的策略请求：谁、在哪台主机、以谁身份、执行哪个程序和哪组参数。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：sudo 不是“把用户变成 root”，而是读取策略后，对一次具体调用请求进行匹配、认证、身份切换和执行。

01明确主体谁在发起请求

02确认位置规则在哪台主机成立

03固定 Runas命令最终以谁运行

04收缩动作绝对路径与参数边界

05校验策略片段、全局与可见授权

06验证边界允许与拒绝矩阵

专题地图

- 知识专题 sudo 的多维策略判定
- 知识专题 从左到右读懂 sudoers
- 知识专题 路径、参数、标签与加载顺序
- 操作专题 推导最小命令集合
- 操作专题 维护 `/etc/sudoers.d`
- 操作专题 `sudo -l`、真实调用与 `sudedit`
- 诊断专题 规则可见但请求不匹配
- 经典任务 Web 运维组最小服务授权

阅读时持续回答

1. 调用者与 Runas 身份分别是谁？
2. 主机字段和命令字段中的 `ALL` 各表示什么？
3. 规则是否固定了真实绝对路径？
4. 命令参数是否仍可选择其他动作？
5. 本次无密码提示来自哪里？
6. 什么证据只证明策略，什么证据才能证明功能？
7. 哪些拒绝路径必须主动测试？

边界：用户与组生命周期归第 07 章；传统权限归第 08 章；Ansible 自动化授权归 RHCE 身份章节。
第 10 章 · 正文

管理员很少真正需要把一个普通用户“变成 root”。更常见的目标是：让值班人员只查看某个服务，让数据库管理员只执行一条备份命令，让应用维护者只编辑一份受控配置。sudo 的价值不在于提供另一条进入 root

Shell 的捷径，而在于把一次特权调用拆成可判定的条件：谁在哪台主机上，准备以哪个目标身份，运行哪个可执行文件和哪组参数。

如果规则只写成 `%ops ALL=(ALL) ALL`，系统确实容易“用起来”，但授权边界几乎消失。相反，若只盯着一行 `sudoers` 语法而不验证真实调用，也会出现另一类错误：`sudo -l` 看得到规则，实际命令却因 `Runas`、路径或参数不匹配而被拒绝；或者命令已经获准执行，但目标服务本身仍然失败。

本章以“把业务要求收缩为可匹配请求”为主线，先建立 `sudoers` 对象模型，再进入配置、查询、验证和诊断。用户与组的创建留给第 07 章，文件权限留给第 08 章，服务内部状态留给第 12 章，自动化授权留给 RHCE 身份章节。

每次读到一条规则，都要重新回答：

谁发起请求？规则在哪台主机成立？命令以谁的身份运行？允许的是哪个程序和哪组参数？是否要求认证？哪些相似调用必须被拒绝？

概念

`sudoers` 规则 是对特权调用请求的策略描述，而不是一个模糊的“管理员角色”。规则把调用者、主机、`Runas` 身份和命令规格关联起来；任何关键维度不匹配，请求都不会命中预期授权。理解这条关系后，`'sudo -l'` 才不只是“有没有权限”的列表，而是后续逐字段核对的证据入口。

概念

主体 是发起 `sudo` 请求的用户、Unix 组或 `'User_Alias'`。主体回答“谁可以提出请求”，不回答命令最终以谁运行。组的创建、附加组生效和账号生命周期归第 07 章；本章只使用已经成立的身份事实。

概念

`Runas` 是命令执行时使用的目标用户和目标组。普通用户 `'alice'` 可以发起请求，而命令以 `'root'`、`'postgres'` 或指定组运行；调用者与 `Runas` 混淆，是“列表有规则但默认调用失败”的高频原因。

概念

主机匹配 决定规则在哪些主机上成立。单机任务常写 `'ALL'`，但这个 `'ALL'` 位于主机字段，并不等于允许所有命令。集中分发同一策略时，主机字段可以继续收缩授权位置；本章不展开 LDAP 或 IdM 策略。

概念

命令规格 由可执行文件路径和可选参数条件组成。绝对路径固定入口程序，参数继续固定程序允许执行的子动作；如果只写路径而不限参数，用户通常可以自行选择参数，因此“参数也是授权边界”是本章最重要的判断。

概念

认证状态 与授权范围是两个维度。`'PASSWD'`、`'NOPASSWD'` 和认证时间戳影响本次是否需要验证调用者密码，却不会自动改变允许的命令集合。一次没有提示密码，既可能来自免密规则，也可能只是已有时间戳尚未失效。

概念

最小特权 不只是“成功命令很少”，还要求相似但未授权的路径、参数、对象和 `root` Shell 被明确拒绝。验收必须同时证明允许矩阵和拒绝矩阵；只要用户仍能通过 Shell、解释器、通配符或可修改脚本选择任意后续动作，边界就没有真正收紧。

操作语义

先建立五个关键入口的接口地图。正文后续只在需要时解释具体操作，不机械重复整份手册页。

`sudo`

SYNOPSIS

```
sudo [-u user] [-g group] [-n] command [argument ...]  
sudo -k
```

按当前 `sudoers` 策略发起一次特权调用，必要时验证调用者身份，并以匹配到的 Runas 凭据执行目标程序。

`-u user` 指定目标 Runas 用户；省略时通常请求默认目标用户。`-g group` 指定目标 Runas 组；必须与规则允许的组匹配。`-n` 非交互模式；需要密码时直接失败，适合验证免密边界。`-k` 使后续需要认证的调用重新验证，避免旧时间戳干扰测试。

`sudo -l`

SYNOPSIS

```
sudo -l  
sudo -l -U user
```

列出当前策略中与调用者相关的 Defaults、Runas 和命令规格，是匹配诊断的起点，不是最终功能验收。

`sudo -l` 当前用户查看自己的可见授权。`-U user` 由 root 或具有相应能力的管理员查看指定用户；不能假设普通用户可任意使用。

`visudo`

SYNOPSIS

```
visudo [-c] [-f sudoers-file]
```

以安全方式编辑 `sudoers`：处理并发锁、保存前解析，并在检查模式下验证配置。

`-f file` 编辑或检查指定 `sudoers` 文件，适合 `/etc/sudoers.d` 片段。`-c` 检查配置；配合 `-f` 检查片段，单独使用时检查完整策略树。

`sudoedit` / `sudo -e`

SYNOPSIS

```
sudoedit file [...]  
sudo -e file [...]
```

让编辑器以调用者身份处理临时副本，再由 `sudo` 受控写回目标文件，避免直接授予通用 `root` 编辑器。

``sudoedit path`` 在 `sudoers` 中作为内建命令书写，不需要前导可执行路径。精确路径只列出需要编辑的文件，并检查父目录不受调用者控制。

``/etc/sudoers.d`` 管理

TYPICAL FLOW

```
visudo -f /etc/sudoers.d/70-webops-httpd
visudo -cf /etc/sudoers.d/70-webops-httpd
visudo -c
```

把本地规则拆成可审计片段；文件名、词法加载顺序、所有者和模式都会影响最终策略。

文件名避免 ``` 和结尾 ``~``；使用等宽零填充编号表达词法顺序。控制权由 `root` 管理，推荐模式 ``0440``；片段通过后仍需全局检查。

[知识专题] `sudo` 的本质：一次多维策略判定

面对“给某用户 `sudo` 权限”的要求，最先要问的不是“在哪一行加 `ALL`”，而是这次工作需要授权哪一类动作。把业务目标改写为可匹配的调用请求后，才能判断哪些字段必须固定、哪些字段可以复用、哪些能力必须明确拒绝。

① [知识点] 调用请求由多个维度共同组成

可把一次 `sudo` 请求抽象为：

```
调用者身份
× 当前主机
× 目标 Runas 用户/组
× 可执行文件路径
× 命令行参数
× 命令标签与 Defaults
× 执行环境
-----
允许或拒绝
```

这解释了为什么“命令名字看起来一样”仍可能不匹配。下面三个调用并不是同一个请求：

```
sudo /usr/bin/systemctl status httpd
sudo /usr/bin/systemctl --no-pager status httpd
sudo -u apache /usr/bin/systemctl status httpd
```

它们的参数或 `Runas` 身份不同。`sudoers` 不是按自然语言理解“都是查看 `httpd`”，而是按规则语义匹配。

② [知识点] 授权、认证、执行和审计是四个阶段

规则加载

- 请求匹配
- 调用者认证
- 切换为 Runas 凭据并构造环境
- 执行目标程序
- 记录允许、拒绝或错误

- 授权回答是否允许这组调用条件。
- 认证通常验证调用者，而不是要求 root 密码。
- 执行由目标程序自己完成；sudo 不能保证服务、文件或网络的业务终态正确。
- 审计记录谁请求了什么以及结果，但日志的完整查询和轮转归第 13 章。

③ [知识点] root Shell 会把有限授权重新扩大为任意授权

允许 `/usr/bin/systemctl restart httpd` 与允许 `/bin/bash` 的安全意义完全不同。后者让用户在 root Shell 中自行选择后续命令，原本针对路径和参数的限制失去作用。因此以下能力通常不应作为最小授权的默认答案：

```
sudo -i
sudo -s
sudo /bin/bash
sudo /bin/sh
sudo 任意通用解释器
```

如果任务确实要求完整系统管理权限，那是另一种授权目标；不能把它包装成“最小特权”。

[Cheatsheet] sudo 不是“变成 root”；先分清调用者与 Runas，再固定主机、路径和参数。授权成功只证明策略允许，不证明目标程序成功。

[知识专题] 从左到右读懂一条 sudoers 规则

一条常见规则可按四段读取：主体、主机、Runas 和命令规格。读取时不要跳过主机字段，也不要吧括号中的身份误认为调用者。

```
%webops ALL=(root:root) /usr/bin/systemctl --no-pager status httpd
```

① [知识点] 主体可以是用户、Unix 组或 User_Alias

```
alice    ALL=(root) /usr/bin/id
%webops  ALL=(root) /usr/bin/systemctl --no-pager status httpd
```

- `alice` 只匹配该用户。
- `%webops` 匹配 Unix 组成员；组的创建和成员生命周期归第 07 章。
- `User_Alias` 可以把多个主体组合为一个可复用名称。

别名名称通常使用大写字母开头的全大写标识，但安全性并不来自“用了别名”，而来自别名中成员以及后续命令范围是否正确。

② [知识点] 主机字段回答规则在哪些主机成立

单机 RHCSA 任务中常见 `ALL`：

```
%webops ALL=(root) /usr/bin/systemctl --no-pager status httpd
```

这里的第一个 `ALL` 是主机匹配，不是命令 `ALL`。在集中分发同一策略的环境中，主机字段可以限制特定主机或 `Host_Alias`。本章只讲本地文件中的匹配模型，不扩展 LDAP 或集中 `sudo` 策略。

③ [知识点] Runas 规格决定目标用户和目标组

```
alice ALL=(root) /usr/bin/id
alice ALL=(postgres) /usr/bin/psql
alice ALL=(root:adm) /usr/bin/id
```

括号中的第一部分是 Runas 用户，冒号后是 Runas 组。省略 Runas 规格时，常见默认目标用户为 `root`，但在阅读现有系统时仍应查看实际 Defaults 和规则，而不是只凭习惯猜测。

调用时可使用：

```
sudo -u postgres /usr/bin/psql
sudo -g adm /usr/bin/id
```

如果规则只允许 `(postgres)`，用户请求默认 `root` 身份时不会命中该授权。

④ [知识点] 命令列表与标签有作用范围

```
%webops ALL=(root) NOPASSWD: /usr/bin/systemctl --no-pager status httpd, \
    PASSWD: /usr/bin/systemctl restart httpd
```

逗号分隔两个命令规格。`NOPASSWD:` 从出现位置开始影响后续命令，直到同一列表中被 `PASSWD:` 等标签改变。为降低误读，复杂规则应换行并在需要切换认证要求的位置显式写出 `PASSWD:` 或 `NOPASSWD:`，不要依赖读者记住远处标签的继承范围。

[Cheatsheet] `%` 是 Unix 组；主机 `ALL`、Runas `ALL` 和命令 `ALL` 位于不同字段，含义不同；标签从出现位置向后生效。

[知识专题] 别名、多个匹配项与最终生效规则

sudoers 允许用别名降低重复，也允许同一用户命中多条规则。这里最危险的误区是认为“更具体的规则一定覆盖更宽泛的规则”。实际调查必须考虑加载顺序和最后匹配结果。

① [知识点] 四类别名分别复用四类对象

```
User_Alias  WEBADMINS = alice, bob
Host_Alias  WEBSERVERS = servera, serverb
Runas_Alias WEBUSERS = root, apache
Cmdnd_Alias HTTPD_READ = /usr/bin/systemctl --no-pager status httpd

WEBADMINS WEBSERVERS=(WEBUSERS) HTTPD_READ
```

- `User_Alias`：调用者集合；
- `Host_Alias`：主机集合；
- `Runas_Alias`：目标身份集合；
- `Cmnd_Alias`：命令规格集合。

别名适合复用稳定集合。只有一条规则时，直接写清楚往往比引入多层别名更容易审计。

② [知识点] 多个匹配项按读取顺序应用

当一个调用者同时匹配多条规则时，不能只找到第一条就停止。发生冲突时，最后匹配值可能决定最终结果，而且“最后”不等于“最具体”。

例如先写精确免密，后面又有宽泛需密码规则：

```
%webops ALL=(root) NOPASSWD: /usr/bin/systemctl --no-pager status httpd
%webops ALL=(root) PASSWD: ALL
```

调查密码行为时必须同时看到两条及其顺序，不能只引用第一条证明 `NOPASSWD`。

③ [知识点] 否定项不适合作为宽泛授权的主要防线

下面的思路看似“允许全部但排除危险命令”：

```
alice ALL=(root) ALL, !/bin/bash
```

这不是可靠的最小授权设计。用户可能通过其他 Shell、解释器、编辑器逃逸、程序子命令或被允许程序的插件机制获得等价能力。优先设计短小允许列表，而不是先给 `ALL` 再尝试列举所有危险例外。

④ [知识点] 规则可读性也是安全控制的一部分

建议每个 `/etc/sudoers.d` 片段围绕一个明确职责：

```
70-webops-httpd
71-dbops-backup
80-helpdesk-account-status
```

同一片段中把业务说明写为注释、把命令按用途分组，并避免把完全无关的团队与命令塞进同一 `Cmnd_Alias`。可读性越差，越容易在后续追加规则时意外放宽边界。

[Cheatsheet] 别名只负责复用；多个匹配要看完整顺序；最小授权优先允许列表，不以“ALL 加否定项”为主结构。

[知识专题] 命令路径和参数共同定义授权动作

`sudoers` 中的命令规格不是一个模糊的“功能名称”。可执行文件路径回答运行哪个程序，参数字符串回答这个程序能执行哪个子动作。最小特权往往主要发生在参数层。

① [知识点] 使用绝对路径减少 PATH 解析歧义

```
alice ALL=(root) /usr/bin/systemctl --no-pager status httpd
```

规则应以目标机上的真实路径为准。编写前先查询：

```
command -v systemctl
readlink -f "$(command -v systemctl)"
```

不要从另一台发行版或旧笔记复制路径。绝对路径只固定了入口程序；若该程序内部再次按 `PATH` 调用子命令，仍要评估执行环境和程序本身的安全边界。路径固定也不等于文件内容不可变，因此还要确认可执行文件、脚本及父目录不受调用者控制。

② [知识点] 未写参数通常表示允许任意参数

```
alice ALL=(root) /usr/bin/systemctl
```

这通常不是“只允许无参数运行 `systemctl`”，而是允许用户自行选择参数，可能覆盖启动、停止、编辑、设置属性等大量动作。若目标是只查看 `httpd`，应把参数写入命令规格：

```
alice ALL=(root) /usr/bin/systemctl --no-pager status httpd
```

若确实要表达“不允许任何参数”，命令规格可在路径后写空参数字符串 `""`。这是一项精确语义，但 RHCSA 常规任务更常见的是把题目要求的实际参数完整写出；无论采用哪种形式，都要在目标机执行允许与拒绝测试。

③ [知识点] 参数必须按规则语义匹配

以下调用在 `sudoers` 匹配上可能不同：

```
/usr/bin/systemctl --no-pager status httpd
/usr/bin/systemctl status --no-pager httpd
/usr/bin/systemctl --no-pager status httpd.service
```

即使目标程序最终可能接受多种等价写法，`sudoers` 仍按命令行参数进行匹配。验收时应使用题目要求的精确调用，并单独测试未授权变式是否被拒绝。

④ [知识点] 通配符会跨越参数边界

参数通配符匹配的是组合后的参数字符串，`*` 可能匹配空白。规则：

```
%logreaders ALL=(root) /usr/bin/cat /var/log/app*
```

可能不只允许读取一个以 `/var/log/app` 开头的文件，还可能匹配追加的第二个敏感路径。参数范围简单时列出精确命令；范围复杂时优先用受控专用工具或经过版本确认的更严格匹配方式，不要把宽泛通配符当作考试捷径。

[Cheatsheet] 绝对路径固定程序；参数固定动作；没有参数限制通常过宽；通配符可匹配空白并跨越参数边界。

[知识专题] Shell、包装脚本和可修改对象为何会扩大权限

有些规则表面上只允许一个文件，实际却把后续命令选择权交给用户。判断安全边界时，不能只看 `sudoers` 一行，还要看被执行程序能否启动其他命令，以及用户能否修改程序、脚本、配置或其父目录。

① [知识点] 通用 Shell 和解释器等于可编程执行入口

以下规则都很难称为有限授权：

```
alice ALL=(root) /bin/bash
alice ALL=(root) /usr/bin/python3
alice ALL=(root) /usr/bin/perl
```

解释器可以读取脚本、执行系统调用并启动其他程序。即使规则尝试固定一部分参数，也要评估参数注入、模块加载和用户可写文件。

② [知识点] root 执行用户可写脚本会把脚本内容变成授权面

```
%ops ALL=(root) /usr/local/bin/restart-web.sh
```

这条规则是否安全，取决于：

- 脚本自身是否由 root 控制
- 父目录是否由普通用户可写
- 脚本调用子命令是否使用安全路径
- 是否读取用户可控配置或环境
- 参数是否会进入 Shell 解释

若 ops 能修改脚本或替换父目录中的文件，sudoers 即使写得很精确，用户仍能把任意内容变成 root 命令。

③ [知识点] 编辑器、分页器和可逃逸程序需要专门评估

许多交互程序支持启动 Shell、读取任意文件、写文件、加载插件或执行外部命令。不能因为命令名称是“查看器”或“编辑器”就默认安全。若目标只是修改某个配置文件，优先考虑 `sudoedit`；若目标只是读取状态，优先使用非交互、固定参数的命令。

④ [知识点] 目录可写性和文件所有权是授权链的一部分

本章不重复第 08、09 章的权限与 ACL 机制，但在授权前必须引用其结论：

```
namei -l /usr/local/bin/restart-web.sh
stat /usr/local/bin/restart-web.sh
getfacl /usr/local/bin/restart-web.sh
```

这些证据用于确认普通用户不能替换执行对象。若发现可写性问题，应先修复对象控制权，而不是通过增加 sudo 规则绕过。

[Cheatsheet] 固定脚本路径不等于固定行为；同时审查解释器、可逃逸程序、脚本内容、父目录和环境输入。

[操作专题] 从业务要求推导最小命令集合

最小授权不是在写完规则后“尽量删一点”，而是在动手前把允许动作和拒绝动作分别写清楚。这个过程能阻止一个常见错误：把“管理 httpd”直接翻译成“允许任意 systemctl”。

① [操作] 把模糊动词改写为可验证动作

原要求：

允许 webops 管理 Web 服务。

需要继续澄清为：

允许: 查看 httpd 状态、重启 httpd
拒绝: 停止、禁用、mask、编辑 unit、操作其他服务、进入 root Shell
目标身份: root
认证: 查看免密, 重启需调用者密码

这些条件会直接映射到命令路径、参数和标签。

② [操作] 在目标系统确认真实命令和调用形式

```
command -v systemctl  
/usr/bin/systemctl --no-pager status httpd  
/usr/bin/systemctl restart httpd
```

先以管理员身份确认目标命令本身有效, 再写 sudoers。不要用 sudo 权限问题掩盖命令拼写、unit 名称或服务故障。

③ [操作] 编写短小、显式的允许列表

```
%webops ALL=(root) NOPASSWD: /usr/bin/systemctl --no-pager status httpd, \  
PASSWD: /usr/bin/systemctl restart httpd
```

这条规则明确:

- 主体为 %webops ;
- 适用于本规则匹配的主机;
- 目标用户为 root;
- 只允许两个参数组合;
- 两个动作的认证要求不同。

④ [操作] 写规则时同步建立拒绝矩阵

至少列出并在验收时执行:

```
sudo /usr/bin/systemctl stop httpd  
sudo /usr/bin/systemctl restart sshd  
sudo /usr/bin/systemctl --no-pager status sshd  
sudo -i  
sudo -s  
sudo /bin/bash
```

预期结果是被策略拒绝。拒绝测试不是“额外安全检查”, 而是证明规则没有超出目标边界的必要证据。

[Cheatsheet] 先写允许/拒绝矩阵, 再写 sudoers; 路径和参数来自目标机; 测试不能只跑成功路径。

[操作专题] 安全维护 `/etc/sudoers.d` 片段

直接使用普通编辑器修改 `/etc/sudoers`，一旦产生语法错误，可能使后续特权访问受阻。`visudo` 的价值不仅是打开编辑器，还包括锁定和语法检查。新规则优先放在独立片段中，便于审计、回退和更新保留。

① [操作] 使用 `visudo -f` 创建或编辑片段

```
visudo -f /etc/sudoers.d/70-webops-httpd
```

保存退出时进行语法检查。不要把 `vim /etc/sudoers.d/...` 作为默认教学路径。

② [操作] 选择会被 `include` 机制读取的文件名

常见 `/etc/sudoers` 包含目录导入指令，例如：

```
@includedir /etc/sudoers.d
```

一些 RHEL 9 系统仍可见兼容写法 `#includedir /etc/sudoers.d`。这里的 `#` 不是普通注释标记，而是旧式 `include` 指令的一部分。维护时以目标机现有 `sudoers` 为准，不要凭笔记替换语法。目录中的文件名不得包含点号，也不得以 `~` 结尾；编辑器备份文件因此通常不会被加载。

推荐使用零填充编号：

```
20-base-operators
70-webops-httpd
90-local-overrides
```

编号帮助表达预期读取顺序，但不能代替检查完整策略树。

③ [操作] 核对所有者和模式

```
chown root:root /etc/sudoers.d/70-webops-httpd
chmod 0440 /etc/sudoers.d/70-webops-httpd
stat -c '%U:%G %a %n' /etc/sudoers.d/70-webops-httpd
```

片段应由可信管理员控制。模式不安全、所有者错误或父目录被篡改都可能使策略失效或形成安全风险。

④ [操作] 先校验片段，再校验完整策略树

```
visudo -cf /etc/sudoers.d/70-webops-httpd
visudo -c
```

`-cf` 适合快速定位当前片段语法；`-c` 才能把 `/etc/sudoers` 和 `include` 片段作为整体检查。一个片段单独通过，不代表别名在完整策略中没有冲突，也不代表后续规则没有改变认证标签。

[Cheatsheet] `visudo -f` 编辑；文件名无 `.`、不以 `~` 结尾；`root` 控制、推荐 `0440`；片段与全局必须分别校验。

[操作专题] 用 `sudo -l` 与真实调用建立分层证据

`sudo -l` 是定位匹配规则的入口，但它不是最终验收。列表输出可能包含 Defaults、Runas 和命令规格；真正调用还会受到路径、参数、认证状态和目标程序自身状态影响。

① [操作] 当前调用者查看自己的授权

```
sudo -l
```

重点读取：

- 哪些 Defaults 适用；
- 允许的 Runas 身份；
- 命令绝对路径；
- 是否显示 NOPASSWD；
- 参数是否与预期一致。

② [操作] 管理员检查指定用户的列表

```
sudo -l -U alice
```

`-U` 不是任何普通用户都能任意使用。默认情况下，应由 `root` 或具备相应授权的管理员执行。若要专门授权“列出他人权限”，`sudoers` 还存在内建 `list` 能力，但这不是 RHCSA 本章的默认配置任务。

③ [操作] 消除认证时间戳对测试的干扰

```
sudo -k
sudo -n /usr/bin/systemctl --no-pager status httpd
```

- `sudo -k` 使下一次需要密码的操作重新认证；
- `sudo -n` 禁止交互提示，若需要密码则直接失败。

因此 `sudo -k` 后的 `sudo -n` 很适合验证某个精确命令是否真正具备 NOPASSWD，但不能证明交互式密码输入流程本身正确。

④ [操作] 同时测试允许和拒绝路径

```
允许命令成功
+ 未授权参数被拒绝
+ 其他服务被拒绝
+ root Shell 被拒绝
```

建议使用目标用户的新登录会话，避免旧会话的组成员关系或环境状态干扰。组刷新机制归第 07 章，本章只要求在验证前确认调用者当前身份正确。

[Cheatsheet] `sudo -l` 看可见授权；`sudo -k` 清认证缓存；`sudo -n` 测非交互；最终必须执行允许与拒绝调用。

[知识专题] NOPASSWD、认证时间戳与非交互执行

“这次没有提示密码”并不能直接证明规则使用了 NOPASSWD。sudo 通常会缓存近期成功认证的状态；测试若不先清除时间戳，很容易得到错误结论。

① [知识点] NOPASSWD 只改变认证要求

```
%webops ALL=(root) NOPASSWD: /usr/bin/systemctl --no-pager status httpd
```

它不允许 `restart`、不允许其他服务，也不自动允许 Shell。命令范围仍由路径和参数决定。

② [知识点] 默认验证的是调用者密码

普通配置下，sudo 要求的是当前调用者自己的密码，而不是 root 密码。这使管理员不必共享 root 凭据，同时仍能把特权调用关联到具体账号。

③ [知识点] 认证时间戳是临时状态

一次成功认证后，后续调用在一段时间内可能不再询问密码。实际期限由配置决定，不能把课件中的某个分钟数当作所有系统的永久默认值。调查时读取目标系统 Defaults，并用 `sudo -k` 建立干净测试条件。

④ [知识点] 非交互任务要显式区分“授权失败”和“需要密码”

自动化或无终端环境常使用：

```
sudo -n <command>
```

它不会等待密码输入。失败可能表示命令未授权，也可能表示命令已授权但需要认证。应结合错误信息、`sudo -l` 和规则标签继续判断，不能仅凭退出非零就扩大为“sudoers 没有加载”。

[Cheatsheet] 无密码提示可能来自时间戳；测试前 `sudo -k`；`NOPASSWD` 改认证，不改命令范围；`-n` 失败要区分未授权与需密码。

[知识专题] 环境清理与 `secure_path` 的必要范围

命令在普通 Shell 中成功、经 `sudo` 后失败，原因不一定是授权。`sudo` 会按策略构造执行环境，`PATH`、`HOME`、编辑器变量和应用变量可能与调用者原环境不同。本章只建立识别和最小调查能力，不穷举所有环境选项。

① [知识点] `env_reset` 降低不受控变量进入特权进程的机会

启用环境重置时，只保留允许的变量并为目标命令构造较受控的环境。这有助于阻止用户通过加载路径、解释器变量或工具配置影响 `root` 进程。

不要为了让一个程序“先跑起来”就默认使用 `sudo -E` 保留整个环境。应先找出目标程序真正需要的变量，并评估其值是否可信。

② [知识点] `secure_path` 可以替换 `sudo` 命令使用的 `PATH`

若配置了 `secure_path`，`sudo` 会使用该值替换调用者的 `PATH`。它有两个直接目的：

- 让管理命令目录在 `PATH` 中可用；
- 降低受限用户通过操纵 `PATH`，使特权脚本调用恶意同名程序的风险。

③ [知识点] `sudoers` 的绝对路径不能自动保护程序内部调用

规则固定 `/usr/local/sbin/backup-app`，但脚本内部若执行：

```
tar -czf ...
```

而不是 `/usr/bin/tar`，行为仍可能依赖 `PATH`。应在脚本中设置安全 `PATH` 或使用绝对路径，并确保脚本和父目录不可被调用者修改。

④ [知识点] 环境诊断要先比较证据再修改 Defaults

```
env | sort
sudo env | sort
sudo -V
sudo -l
```

`sudo -V` 展示的详细策略信息通常需要管理员权限才能完整查看。比较差异后，只对明确需要的变量做最小配置；不要全局放宽 `env_keep` 或授予 `SETENV` 来掩盖单个应用的配置问题。

[Cheatsheet] 经 `sudo` 后环境可能不同；`secure_path` 影响 `PATH`；绝对入口不保证脚本内部安全；先比较环境，再做最小修改。

[操作专题] 用 `sudoedit` 授权受控文件编辑

直接允许 `sudo vim` 往往比预期更强：编辑器可能打开其他文件、写入任意路径或启动 Shell。`sudoedit` 的模型是把目标文件复制到临时位置，让编辑器以调用者身份运行，编辑完成后再由 `sudo` 受控写回。

① [操作] 在 `sudoers` 中把 `sudoedit` 写成内建命令

```
%webops ALL=(root) sudoedit /etc/httpd/conf.d/site.conf
```

`sudoedit` 是 `sudo` 内建能力，在 `sudoers` 中不应写前导路径。调用者执行：

```
sudoedit /etc/httpd/conf.d/site.conf
# 或
sudo -e /etc/httpd/conf.d/site.conf
```

② [操作] 只列出确实需要编辑的路径

```
%webops ALL=(root) sudoedit /etc/httpd/conf.d/site.conf, \
                        sudoedit /etc/httpd/conf.d/tls.conf
```

避免直接允许整个 `/etc` 或使用宽泛通配符。若文件集合经常变化，先确认是否应改为受控部署流程，而不是持续扩大人工编辑范围。

③ [操作] 检查目标文件父目录是否由调用者可写

```
namei -l /etc/httpd/conf.d/site.conf
stat /etc/httpd/conf.d/site.conf
getfacl /etc/httpd/conf.d/site.conf
```

不得把 `sudoedit` 授权给位于调用者可写目录中的文件。否则调用者可能通过替换文件或链接改变真正被编辑的对象。现代 `sudo` 有额外防护，但安全设计仍应先确保路径控制权正确。

④ [操作] 验证编辑能力而不扩大为通用 `root` 编辑器

验收应证明：

- 指定文件能编辑；
- 未授权文件被拒绝；
- `sudo vim /etc/shadow` 被拒绝；
- 编辑器进程以调用者身份运行；
- 写回后的目标文件仍保持预期所有者和权限。

[Cheatsheet] sudoers 中写 `sudoedit`，不写 `/usr/bin/sudoedit`；按文件精确授权；父目录不可由调用者写；拒绝通用 root 编辑器。

[诊断专题] `sudo -l` 看得到规则，实际命令却不匹配

这个症状说明“至少有一条规则与用户相关”，但不等于当前请求的每个维度都匹配。诊断时不要立即追加 `ALL`，而应把列表中的命令规格和真实调用逐字段比较。

① [诊断] 先记录列表和真实调用

```
sudo -l
printf '%q ' /usr/bin/systemctl status httpd.service; echo
```

保留：调用者、Runas、可执行路径、每个参数及顺序。不要只记录自然语言“运行了 systemctl”。

② [诊断] 核对 Runas 身份

```
sudo /usr/bin/id
sudo -u apache /usr/bin/id
```

若规则只允许 `(apache)`，默认 root 请求会失败；若规则只允许 root，以 `-u apache` 调用同样不会命中。

③ [诊断] 核对路径、参数和值

列表允许：

```
(root) /usr/bin/systemctl --no-pager status httpd
```

实际调用：

```
sudo /usr/bin/systemctl status httpd.service
```

差异包括缺少 `--no-pager`、unit 参数不同，可能还有参数顺序不同。下一条最有区分度的证据不是重新安装 sudo，而是按列表中的精确形式调用。

④ [诊断] 检查后续规则和标签

```
visudo -c
sudo -l
```

同时阅读 `/etc/sudoers` 和 `/etc/sudoers.d` 中与主体相关的所有规则。若命令获准但密码行为与预期不同，重点查看后续 `PASSWD` / `NOPASSWD` 匹配，而不是只看第一条规则。

⑤ [诊断] 最小修复后同时复测允许和拒绝路径

修复选择只有两类：

- 调用者改用已授权的精确形式；
- 业务确实需要另一种形式时，增加另一条精确命令规格。

不要把规则改成 `/usr/bin/systemctl *`。修复后重跑原允许命令、原失败变式以及其他服务的拒绝测试。

[Cheatsheet] 列表可见不等于请求匹配；逐项比 Runas、路径、参数和顺序；修复精确差异，不扩大为通配符。

[诊断专题] 片段存在，但没有加载或行为与预期不同

这类故障通常落在三层：文件未进入 include、完整策略存在冲突、或认证与环境被误判。按最有区分度的证据推进，避免反复重写同一行规则。

① [诊断] 检查文件名、路径和元数据

```
ls -ld /etc/sudoers.d
ls -l /etc/sudoers.d
stat /etc/sudoers.d/70-webops-httpd
```

重点排除：

- 文件名包含 `.`；
- 文件名以 `~` 结尾；
- 写到了未被 include 的目录；
- 所有者或模式异常；
- 文件实际是编辑器临时文件。

② [诊断] 片段通过后继续检查全局

```
visudo -cf /etc/sudoers.d/70-webops-httpd
visudo -c
```

片段语法错误会直接阻断；片段通过而全局失败，则继续处理别名、其他片段或主文件中的问题。

③ [诊断] 区分免密规则与认证时间戳

```
sudo -k
sudo -n <exact-command>
```

- 精确命令成功：可支持 NOPASSWD 判断；
- 提示需要密码：命令可能获准但要求认证；
- 显示不允许执行：继续查匹配维度。

④ [诊断] 区分授权失败与环境导致的程序失败

若 sudo 已经启动程序，但程序报告“command not found”、找不到配置或缺少变量，应比较环境和 `secure_path`。不要因为目标程序失败就把 `sudoers` 改为 `ALL`。

[Cheatsheet] 先文件名和元数据，再片段和全局语法，再认证状态，最后区分环境与目标程序故障。

[诊断专题] sudo 已允许，但目标操作仍然失败

看到密码验证成功或 sudo 没有拒绝，不代表业务任务完成。目标程序可能因自身配置、文件权限、SELinux、服务依赖或输入错误而退出。本章只负责把故障边界分清，再转入自然归属章节。

① [诊断] 先确认策略层已经通过

证据包括：

- `sudo -l` 中存在匹配命令；
- 调用未出现 `sudoers` 拒绝信息；
- 目标程序已经输出自己的错误；
- 日志中的执行命令和 `Runas` 身份符合预期。

② [诊断] 读取目标程序自己的退出状态和错误

```
sudo <exact-command>
printf 'rc=%s\n' "$?"
```

不要只截取“sudo 成功认证”部分。参数错误、对象不存在和服务失败都由目标程序返回。

③ [诊断] 按对象归属转入相邻章节

- `systemctl` 已执行但 `unit` 失败：第 12 章；
- 目标文件仍不可写：第 08、09 章；
- `AVC` 拒绝：第 28、29 章；
- 日志查询和持久性：第 13 章。

本章保留接口，不复制相邻章节的完整诊断。

④ [诊断] 不用扩大授权掩盖目标程序问题

目标服务启动失败时，把规则从精确命令改为 `ALL` 不会修复服务配置，只会增加新的风险。最小修复必须作用于真正失败的对象，之后再从 `sudo` 调用入口完整复测。

[Cheatsheet] `sudo` 通过只证明策略允许；读取目标程序退出状态；故障转入自然章节；不要用更宽授权替代修复。

[经典任务] 为 Web 运维组建立可验证的最小服务授权

环境与当前状态

主机 `servera` 上已经存在 Unix 组 `webops`，用户 `alice` 是该组成员。`httpd` 已安装，管理员已确认以下命令在 `root` 身份下本身有效：

```
/usr/bin/systemctl --no-pager status httpd
/usr/bin/systemctl restart httpd
```

系统中当前存在一条过宽规则：

```
%webops ALL=(root) NOPASSWD: ALL
```

目标终态

1. 删除或替换上述过宽规则。
2. `webops` 成员只允许：
 - 免密查看 `httpd` 状态；
 - 输入调用者自己的密码后重启 `httpd`。
3. 不允许停止、禁用或 `mask httpd`。
4. 不允许操作其他 `unit`。
5. 不允许 `sudo -i`、`sudo -s`、Shell、解释器或通用 `root` 编辑器。
6. 新规则存入 `/etc/sudoers.d/70-webops-httpd`。
7. 不修改无关 `sudoers` 内容。

限制条件

- 必须使用 `visudo` 维护规则；
- 不得加入 `wheel`；
- 不得使用命令 `ALL` 或参数通配符；
- 本任务没有 RHEL 9 live VM，下面命令是推荐验证流程，不声称已实测。

验收证据

```
片段语法
→ 完整策略树
→ alice 可见规则
→ status 免密允许
→ restart 非交互失败、交互认证后允许
→ 其他动作与 root Shell 被拒绝
→ httpd 结果由 systemd 证据另行确认
```

[参考解答] 为 Web 运维组建立可验证的最小服务授权

① 调查现有规则和调用者状态

```
id alice
grep -Rns -- '%webops' /etc/sudoers /etc/sudoers.d 2>/dev/null
sudo -l -U alice
```

确认 `alice` 当前确实属于 `webops`，并定位过宽规则的真实文件。不要在未知来源上再追加一条更具体规则后就假设它会覆盖 `ALL`；读取顺序和最后匹配会影响结果。

② 使用 `visudo` 创建目标片段

```
visudo -f /etc/sudoers.d/70-webops-httpd
```

写入：

```
# webops: read httpd state without a password; restart with caller authentication.
%webops ALL=(root) NOPASSWD: /usr/bin/systemctl --no-pager status httpd, \
    PASSWD: /usr/bin/systemctl restart httpd
```

随后删除或收缩旧的 `%webops ... NOPASSWD: ALL`。不能保留宽泛规则并指望精确规则自动形成上限。

③ 核对文件控制权并校验配置

```
chown root:root /etc/sudoers.d/70-webops-httpd
chmod 0440 /etc/sudoers.d/70-webops-httpd
stat -c '%U:%G %a %n' /etc/sudoers.d/70-webops-httpd
visudo -cf /etc/sudoers.d/70-webops-httpd
visudo -c
```

`visudo -cf` 证明片段可解析；`visudo -c` 证明当前完整策略树通过语法与结构检查。两层都需要。

④ 以 alice 的新登录会话查看授权

```
su - alice
id
sudo -l
```

预期列表只出现两个精确 `systemctl` 调用，不出现 `(root) ALL`、Shell 或其他服务。

⑤ 验证免密状态命令

```
sudo -k
sudo -n /usr/bin/systemctl --no-pager status httpd
printf 'rc=%s\n' "$?"
```

`sudo -k` 清除认证时间戳；`-n` 禁止提示密码。若精确 `status` 命令成功，才能支持“该调用具备 NOPASSWD”的判断。

⑥ 验证重启仍要求认证

先进行非交互测试：

```
sudo -k
sudo -n /usr/bin/systemctl restart httpd
printf 'rc=%s\n' "$?"
```

预期因需要密码而失败，而不是直接执行。随后交互执行：

```
sudo /usr/bin/systemctl restart httpd
```

输入的是 `alice` 自己的密码。`sudo` 调用成功后，再用 `systemd` 层的证据确认服务状态；这部分对象模型归第 12 章。

⑦ 验证拒绝矩阵

```
sudo /usr/bin/systemctl stop httpd
sudo /usr/bin/systemctl disable httpd
sudo /usr/bin/systemctl restart sshd
sudo /usr/bin/systemctl --no-pager status sshd
sudo /usr/bin/systemctl status httpd.service
sudo -i
sudo -s
sudo /bin/bash
```

所有调用都应被拒绝。特别注意：`status httpd.service` 即使对 `systemctl` 来说可能指向同一 unit，也不是规则中授权的精确参数字符串。

⑧ 典型错误

- 保留旧 `%webops ... ALL`，导致拒绝测试仍可通过其他规则执行；
- 只执行 `visudo -cf`，未执行完整 `visudo -c`；
- `status` 测试前未执行 `sudo -k`，把认证时间戳误判为 `NOPASSWD`；
- 只验证两个允许命令，没有测试其他服务和 `root Shell`；
- 把 `sudo` 成功扩大为 `httpd` 功能健康。

[经典任务] 诊断可见规则与实际命令不匹配

已知证据

用户 `alice` 执行 `sudo -l` 时看到：

```
(root) /usr/bin/systemctl --no-pager status httpd
```

但她执行：

```
sudo /usr/bin/systemctl status httpd.service
```

系统拒绝该调用。管理员准备把规则改为：

```
alice ALL=(root) /usr/bin/systemctl *
```

任务要求

1. 解释为什么列表可见但真实调用仍不匹配；
2. 给出下一条最有区分度的验证命令；
3. 提供不扩大为通配符的最小修复方案；
4. 设计允许与拒绝测试。

[参考解答] 诊断可见规则与实际命令不匹配

① 比较规则与请求

规则要求的参数字符串是：

```
--no-pager status httpd
```

实际请求是：

```
status httpd.service
```

至少存在两处差异：缺少 `--no-pager`，最后一个参数由 `httpd` 变为 `httpd.service`。`sudo -l` 证明 `alice` 拥有一条相关授权，不证明任意 `systemctl status` 形式都会命中。

② 执行最有区分度的精确调用

```
sudo /usr/bin/systemctl --no-pager status httpd
```

若这条成功，说明规则加载、主体、主机、Runas 和路径大体成立，故障集中在参数匹配。若仍失败，再检查 `sudo -l` 中 Runas、后续规则和完整策略。

③ 选择最小修复

若业务没有要求 `httpd.service` 形式，直接让调用者使用已经授权的精确命令，不修改规则。

若业务明确要求两种形式，可增加第二条精确命令：

```
alice ALL=(root) /usr/bin/systemctl --no-pager status httpd, \
    /usr/bin/systemctl status httpd.service
```

不应使用：

```
alice ALL=(root) /usr/bin/systemctl *
```

因为它可能允许 `stop`、`disable`、`edit`、`mask` 以及其他 `unit`。

④ 重新验证

允许路径：

```
sudo /usr/bin/systemctl --no-pager status httpd
sudo /usr/bin/systemctl status httpd.service # 仅在第二种形式确有业务需要并已精确加入时
```

拒绝路径：

```
sudo /usr/bin/systemctl stop httpd
sudo /usr/bin/systemctl restart sshd
sudo /usr/bin/systemctl edit httpd
sudo -i
```

最后执行 `visudo -c`，并检查 `sudo -l` 不出现意外宽泛规则。

[本章收束] 用证据证明最小特权，而不是只写出一行规则

本章的工作方法可以压缩为一条可重复执行的链路：

把业务动词改写为允许动作和拒绝动作

→ 确认调用者、主机、Runas、可执行路径与参数

→ 用 visudo 维护独立片段

→ 校验片段和完整策略树

→ 用 sudo -l 读取可见授权

→ 清除认证时间戳后测试认证边界

→ 同时执行允许矩阵与拒绝矩阵

→ 把目标程序故障转入自然归属章节

规则设计阶段关注的是“能不能命中”；验收阶段还要关注“有没有意外命中”。因此，最小特权的证据必须同时包含成功路径和失败边界。只要仍能进入 root Shell、操作其他对象、改变未授权参数，或者通过用户可写脚本改变实际行为，就不能把规则称为最小授权。

主要判断表

看到的现象	当前最多能证明什么	下一条有区分度的证据
<code>visudo -cf</code> 通过	当前片段可被解析	<code>visudo -c</code> 检查完整策略树
<code>visudo -c</code> 通过	当前完整策略语法和引用关系可接受	以目标调用者执行 <code>sudo -l</code>
<code>sudo -l</code> 出现命令	存在相关可见授权	按列表中的 Runas、路径和参数精确调用
调用未被 sudoers 拒绝	策略层允许启动目标程序	读取目标程序错误、退出状态和功能证据
本次没有提示密码	可能是 NOPASSWD，也可能有认证时间戳	<code>sudo -k</code> 后执行 <code>sudo -n <精确命令></code>
精确允许命令成功	成功路径成立	继续测试其他参数、其他对象和 root Shell 被拒绝
经 sudo 后找不到子命令	可能是环境或 secure_path 差异	比较 <code>env</code> 、 <code>sudo env</code> 和脚本内部路径
<code>systemctl</code> 已运行但服务失败	sudo 授权链已推进到目标程序	转入第 12 章检查 unit、日志和服务功能

章末 Cheatsheet

```
# 安全编辑片段
visudo -f /etc/sudoers.d/70-webops-httpd

# 片段与完整策略校验
visudo -cf /etc/sudoers.d/70-webops-httpd
visudo -c

# 查看授权
sudo -l
sudo -l -U alice

# 清除认证时间戳并做非交互测试
sudo -k
sudo -n <exact-command>

# 指定 Runas 身份
sudo -u postgres <command>
sudo -g adm <command>

# 受控编辑
sdoedit /etc/httpd/conf.d/site.conf
```

安全边界：避免命令 `ALL`、参数宽通配符、通用 Shell、解释器、可逃逸编辑器，以及由调用者可修改的脚本或父目录。环境问题先比较普通环境与 `sudo` 环境，不要默认全局保留变量。

向下一章交接

本章把“是否允许启动某个特权命令”解释清楚。下一章《进程、作业、信号与调度优先级》接手命令启动后的运行实例：如何识别 PID、判断进程状态、控制前后台作业、发送信号并调整调度优先级。不要把 `sudo` 授权成功误当成进程生命周期已经得到正确管理。